

Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 5 — The Cheat Sheet

This is not a worksheet. There is nothing to fill in and nothing to work out.

It is the sheet you keep open *beside* the Programming 5 packet. Everything is already answered. When you get stuck, the rule is on here — look it up rather than trying to work it out.

The one idea everything else sits on

A string is a row of numbered boxes, and the numbering starts at 0.

That is it. That is the whole idea. Every operation below is just a different way of pointing at some of the boxes.

```
subject = "ComputerScience"
```

C	o	m	p	u	t	e	r	S	c	i	e	n	c	e
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Fifteen characters, numbered 0 to 14. Because the count starts at 0, the *last* box is always one less than the length. Fifteen characters, last box 14.

A space is a box too. So is a full stop, and so is an exclamation mark. "Count every character!" is 22 boxes, not 19.

The six operations, with what each one actually does

Read the middle column as *a sentence about what happens*. The example beside it is the same thing, done.

You type	What it does	Example	Comes out
<code>len(s)</code>	Counts the boxes. Gives you a <i>number</i> .	<code>len(subject)</code>	15
<code>s[n]</code>	Hands you the <i>one</i> character sitting in box <i>n</i> .	<code>subject[0]</code>	C
<code>s[:n]</code>	Everything <i>from the start</i> up to box <i>n</i> , not including <i>n</i> .	<code>subject[:4]</code>	Comp
<code>s[-n:]</code>	The <i>last</i> <i>n</i> characters. The minus means “count from the back”.	<code>subject[-3:]</code>	nce
<code>s[a:b]</code>	The middle piece: start at box <i>a</i> , stop <i>before</i> box <i>b</i> .	<code>subject[3:8]</code>	puter
<code>s.upper()</code>	Gives back a <i>shouted copy</i> . The original is untouched.	<code>subject.upper()</code>	COMPUTER-SCIENCE

And one that is not slicing at all — joining. The plus sign glues two strings end to end. It puts nothing in between, so if you want a space you type the space.

```
print("Hello" + "Everyone")      # HelloEveryone    <- 13 characters, no gap
print("Hello" + " " + "Everyone") # Hello Everyone  <- the space is yours to add
```

Two things that look the same and are not

One number in the brackets gives you a *character*. Two numbers give you a *string*. That is the difference, and it catches everybody once.

Code	Comes out	Why
<code>subject[3]</code>	p	One box. One character.
<code>subject[3:4]</code>	p	Boxes 3 up to <i>but not including</i> 4 — so still just box 3.
<code>subject[15]</code>	an error	There is no box 15. The last one is 14.
<code>subject[10:99]</code>	ience	Slicing past the end is <i>allowed</i> . It just stops at the end.

Read those last two rows together, because they are the useful pair. Asking for *one* box that does not exist is an error. Asking for a *range* that runs off the end is fine — Python quietly gives you whatever was there. That is why `"Ng"[:3]` gives Ng rather than breaking.

How to actually build a program out of these

This is the part the packet asks you to work out. Here it is, done, so you can copy the **method**.

The method, and it never changes:

1. Write **one** line.
2. `print()` it and press Run.
3. Look at what came out. Is it what you expected?
4. Only then write the next line.

Never write five lines and then run. If it breaks you will not know which of the five did it. One line, run, look. Every time.

The job: make a nickname. Take the first three letters of somebody's surname, put them in capitals, and stick their year group on the end.

So **Rawson, Year 11** should come out as RAW11.

Four operations, one at a time. Watch what each line prints.

Step 1 — get the two things you need, and prove you have them.

```
surname = input("Surname: ")
year = input("Year group: ")
print(surname)
print(year)
```

Type Rawson and 11. It prints Rawson then 11. That looks like it does nothing useful — and that is exactly the point. You now *know* the two variables hold what you think they hold.

Step 2 — cut the first three letters off the surname.

```
print(surname[:3])
```

Prints Raw. Boxes 0, 1 and 2 — start of the string up to but not including box 3. *Run it before you go on.*

Step 3 — shout it.

```
print(surname[:3].upper())
```

Prints RAW. You have added `.upper()` onto the end of the piece you already had. Reading left to right: take the surname, cut the first three letters, then shout the result.

Step 4 — join the year group on.

```
nickname = surname[:3].upper() + year
print(nickname)
```

Prints RAW11. Done. Notice we stored it in a variable called `nickname` this time, instead of printing it straight out — because now we have something worth keeping.

Step 5 — the one that will bite you.

`input()` always hands back **text**, even when you typed a number. So `year` is already the string `"11"` and the `+` works.

But if the year group came from somewhere else — a calculation, say — it would be a *number*, and joining text to a number is an error. You cast it first:

```
year = 11 # a number this time, not typed in
nickname = surname[:3].upper() + str(year) # str() turns 11 into "11"
print(nickname)
```

Still prints `RAW11`. `str()` is the whole reason casting exists: it makes a number safe to glue onto text.

Check it against these before you call it finished.

Surname	Year	Should give	Why this one is here
Rawson	11	RAW11	The ordinary case.
Lovelace	9	LOV9	A one-digit year.
Hopper	13	HOP13	Two digits again.
Ng	11	NG11	Only two letters. Slicing past the end does not break — it just gives you what is there.

When it goes wrong — what the red text is telling you

An error message is not the computer telling you off. It is telling you *which line* and *what kind of mistake*. These four are the ones you will meet on this topic.

What it says	What that means	What to do
IndexError: string index out of range	You asked for a box that does not exist.	Count again. The last box is <code>len(s) - 1</code> , not <code>len(s)</code> .
TypeError: can only concatenate str (not "int") to str	You tried to glue a number onto text with <code>+</code> .	Wrap the number in <code>str()</code> .
AttributeError: 'int' object has no attribute 'upper'	You used a <i>text</i> operation on a <i>number</i> .	Check what that variable is actually holding. It is not a string.
SyntaxError: unexpected EOF while parsing	A bracket is open and never closed.	Count the <code>(</code> and the <code>)</code> . They must match.

And one that gives no error at all, which is worse.

```
print(name.upper)      # forgot the brackets
```

This prints something like `<built-in method upper of str object at 0x104...>`. No red text, no crash — just nonsense. **It means you named the operation without running it.** Put the brackets on: `name.upper()`.

Three habits that fix most of it

1. Put `print()` around everything.

In the browser editor, a line on its own shows you *nothing at all* — no answer and no error. Typing `subject[:4]` looks like it failed. It did not; you simply never asked to see it. `print(subject[:4])` shows you Comp.

2. Build one line at a time, and run after each one.

Four working lines and one broken one is a five-second fix. Five lines written blind is a puzzle.

3. When you are stuck, print the thing just before the stuck bit.

If `surname[:3].upper()` is not doing what you expect, print `surname` on its own first. Nine times out of ten the variable is not holding what you assumed.

The exam writes it differently — the translation

You **type** Python. The exam paper **shows** you OCR's own language, ERL. Same ideas, different words. You are never asked to type ERL — only to read it.

ERL (what you read)	Python (what you type)	Gives
<code>subject.length</code>	<code>len(subject)</code>	15
<code>subject.left(4)</code>	<code>subject[:4]</code>	Comp
<code>subject.right(3)</code>	<code>subject[-3:]</code>	nce
<code>subject.substring(3,5)</code>	<code>subject[3:8]</code>	puter
<code>subject.upper</code>	<code>subject.upper()</code>	COMPUTERSCIENCE
<code>ASC('A')</code>	<code>ord('A')</code>	65
<code>CHR(97)</code>	<code>chr(97)</code>	a

The single most important row is substring, and it is the odd one out.

ERL's `substring(3,5)` means *start at 3, take 5 characters*. Python's `[3:8]` means *start at 3, stop before 8*. **One counts how many, the other says where to stop.** They land in the same place here, which is why it is worth noticing rather than assuming.

ERL: start, how many. Python: start, stop. If you remember one line off this sheet, make it that one.

Keep this sheet. It works for the Programming 5 worksheet, the activities, and the username challenge — and every string question you will meet after them.